Lecture 11: Gradients of expectations, ML vocabulary, and research workflows

COMP 521 • Probability, Optimization, and Scientific Discovery • Sections 5.5–5.6 and 6.1–6.3

This handout is one part of the full COMP 521 notes. Section, equation, figure and listing numbers match the full notes, and references to anything outside this handout point there. The notation table is in "How to use these notes" (Lecture 1).

5.5 Randomness and differentiability

To differentiate an expectation $\mathbb{E}_{X\sim p_\theta}[f(X)]$ with respect to distribution parameters, write the random variable as a smooth function of parameter-free noise. For a Gaussian, $X = \mu + \sigma\epsilon$ with $\epsilon \sim \mathcal{N}(0,1)$, so

$$\frac{\partial}{\partial \mu}\mathbb{E}_\epsilon\big[f(\mu+\sigma\epsilon)\big] = \mathbb{E}_\epsilon\big[f'(\mu+\sigma\epsilon)\big] \tag{41}$$

whenever differentiation and expectation can be exchanged, which holds when f is continuous and differentiable almost everywhere with a suitably bounded derivative. This is the *reparameterization* or *pathwise* estimator [1]. In JAX, fix the random key (or the draws) and call `jax.grad` on the sample average, as in Listing 1. Holding draws fixed while comparing nearby parameters is also called using *common random numbers*; it removes sampling noise from the comparison.

In Jaxstro: differentiating with respect to the IMF slope

Inverse-transform sampling (Eq. (6)) is a reparameterization: $M = F^{-1}(U;\gamma)$ is smooth in γ for fixed U. So gradients of population statistics with respect to γ come from `jax.grad` through a Progenax-style sampler with fixed uniforms, as long as the statistic is a smooth function of the masses.

ex02_imf_pathwise.py -- pathwise IMF gradients: a smooth statistic and an indicator

```python
"""Inverse-CDF IMF sampling as a reparameterization, and where pathwise gradients fail.

p(m | gamma) proportional to m^(-gamma) on [m_lo, m_hi]  (single power law, gamma != 1).
"""
import jax
import jax.numpy as jnp

jax.config.update("jax_enable_x64", True)

m_lo, m_hi, m_crit = 0.5, 100.0, 8.0  # Msun; m_crit = toy core-collapse threshold


def inv_cdf(u, gamma):  # M = F^{-1}(U; gamma): differentiable in gamma for fixed u
    k = 1.0 - gamma
    return (m_lo**k + u * (m_hi**k - m_lo**k)) ** (1.0 / k)


def mean_mass_exact(gamma):
    k = 1.0 - gamma
    return (k / (k + 1.0)) * (m_hi**(k + 1) - m_lo**(k + 1)) / (m_hi**k - m_lo**k)


def frac_above_exact(gamma):  # P(M > m_crit)
```

```
    k = 1.0 - gamma
    return (m_hi**k - m_crit**k) / (m_hi**k - m_lo**k)


u = jax.random.uniform(jax.random.key(1), (400_000,))  # fixed draws
gamma = 2.35  # Salpeter-like slope


def mean_mass_mc(g):
    return jnp.mean(inv_cdf(u, g))


def frac_above_mc(g):  # an indicator: piecewise constant in gamma for fixed u
    return jnp.mean((inv_cdf(u, g) > m_crit).astype(jnp.float64))


def frac_above_smooth(g, width=0.25):  # sigmoid relaxation: biased but differentiable
    return jnp.mean(jax.nn.sigmoid((inv_cdf(u, g) - m_crit) / width))


if __name__ == "__main__":
    print(f"E[M]        MC = {mean_mass_mc(gamma):.4f}  exact = {mean_mass_exact(gamma):.4f}")
    print(f"dE[M]/dg    pathwise = {jax.grad(mean_mass_mc)(gamma):+.4f}"
          f"  exact = {jax.grad(mean_mass_exact)(gamma):+.4f}")
    print(f"P(M>8)      MC = {frac_above_mc(gamma):.5f}  exact = {frac_above_exact(gamma):.5f}")
    print(f"dP/dg       pathwise = {jax.grad(frac_above_mc)(gamma):+.5f}"
          f"  exact = {jax.grad(frac_above_exact)(gamma):+.5f}")
    print(f"dP/dg       sigmoid-relaxed = {jax.grad(frac_above_smooth)(gamma):+.5f}")
```

Output (executed for these notes):

```
E[M]        MC = 1.6308  exact = 1.6279
dE[M]/dg    pathwise = -1.8547  exact = -1.8513
P(M>8)      MC = 0.02280  exact = 0.02292
dP/dg       pathwise = +0.00000  exact = -0.06166
dP/dg       sigmoid-relaxed = -0.06136
```

For the mean mass, the pathwise gradient agrees with the exact one to Monte Carlo accuracy. For the fraction of stars above $8\,M_\odot$, it returns **zero**, while the true derivative is -0.062: a steeper IMF makes fewer massive stars. For fixed uniforms the indicator $\mathbb{1}[F^{-1}(u;\gamma) > 8]$ is a step function of γ (Fig. 10). The missing contribution is the Leibniz boundary term, the rate at which probability mass crosses $m = 8\,M_\odot$ as γ changes, and Eq. (41) does not apply because f is discontinuous.

Remedies, in order of preference:

1. **Integrate the discontinuity analytically** where you can: here, use the CDF, $P(M > 8) = 1 - F_M(8;\gamma)$, which is smooth in γ.
2. **Score-function (likelihood-ratio) estimator:** $\nabla_{\boldsymbol{\theta}} \mathbb{E}_{p_{\boldsymbol{\theta}}}[f(X)] = \mathbb{E}_{p_{\boldsymbol{\theta}}}[f(X) \nabla_{\boldsymbol{\theta}} \log p_{\boldsymbol{\theta}}(X)]$ when f does not itself depend on $\boldsymbol{\theta}$. Unbiased for discontinuous f and discrete X, but often high-variance; subtract a baseline to reduce it.
3. **Smooth relaxation:** replace the step by a sigmoid of width w. Differentiable but *biased* by an amount that depends on w. For $w = 0.25\,M_\odot$ the relaxed derivative, computed by quadrature, is -0.0619 against the exact -0.0617, a 0.3% bias that is smaller than the Monte Carlo error of the listing's estimate (-0.0614 ± 0.0012). Report w and test the sensitivity of conclusions to it.

Remnant type, detection above a flux limit, and membership in a colour cut are all indicators. Each needs one of these treatments before its gradient means anything.

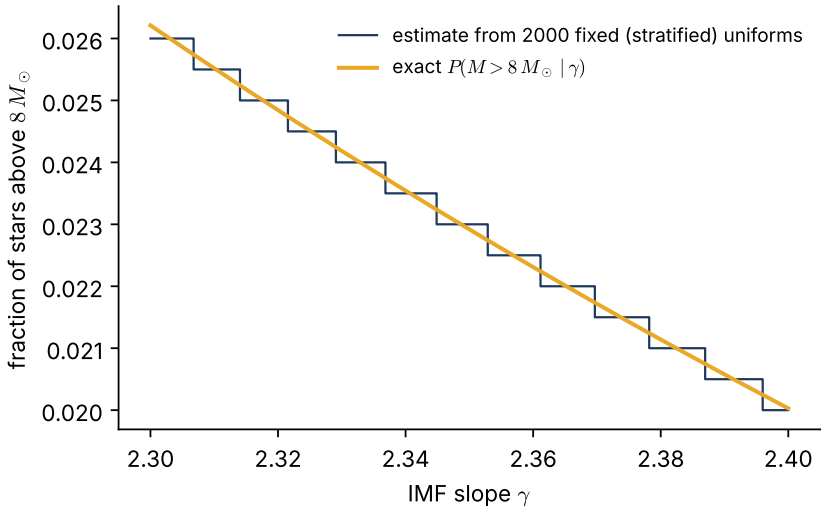


Figure 10: With the uniforms held fixed, the estimated fraction of stars above $8\,M_\odot$ is piecewise constant in γ, so its pathwise derivative is zero almost everywhere. Stratified uniforms are used here to remove sampling noise and show the steps.

5.6 Parameters, hyperparameters, and training jargon

A *physical parameter* controls the astrophysical model (a mixing efficiency). A *model weight* is a fitted coefficient of a surrogate (a neural-network weight). A *hyperparameter* controls a fitting procedure or a level of a hierarchical model (a learning rate, a GP length scale, a prior width). The same quantity can play different roles at different levels of a hierarchy, so say which you mean.

A *batch* is a set of examples processed together; a *mini-batch* is a subset used for one update; *stochastic gradient descent* estimates the full gradient from a mini-batch. *Momentum* accumulates past gradients. *Gradient clipping* caps gradient norms to stabilize training.

The *training set* updates weights. The *validation set* chooses architectures, regularization and when to stop (*early stopping*). The *test set* is inspected once, after every choice is frozen. A *metric* is what you report (maximum relative luminosity error, energy drift); it need not be the loss you trained on. *Overfitting* learns quirks of the training data; *underfitting* misses real structure; *generalization* is performance on new cases; *distribution shift* is a mismatch between training and deployment inputs; *out-of-distribution* inputs lie outside the training support, where emulators fail without warning.

Leakage between neighbouring timesteps

Splitting timesteps of the same tracks at random between training and test sets puts near-duplicates of every test state in the training set. The test error then measures interpolation along tracks the network has already seen. Hold out whole tracks, and where the science requires it, whole regions of mass and metallicity or whole evolutionary phases.

Calibration has two meanings. *Physical* calibration fits a model to reference data. *Probabilistic* calibration asks whether stated probabilities match frequencies over many cases, which is what Section 3.7 tests. Say which one you mean.

6 Two research workflows

The tools of Sections 1 to 5 become research only when combined in the right order. Two templates follow. Each step names the section that supplies its tools and the failure it guards against.

6.1 Workflow 1: how does uncertain opacity affect the TRGB?

Rather than one scalar opacity multiplier, parameterize a smooth, correlated perturbation of the opacity table:

$$\log \kappa(T, \rho, \boldsymbol{X}) = \log \kappa_0(T, \rho, \boldsymbol{X}) + \sum_{k=1}^{K} a_k \, b_k(T, \rho, \boldsymbol{X}), \tag{42}$$

with reference opacity κ_0 (here κ is the opacity, not a condition number), basis functions b_k over thermodynamic conditions, and uncertain coefficients a_k. Exponentiation keeps κ positive. The prior covariance of $\boldsymbol{a}$ should come from comparisons between independent opacity calculations or published error estimates; a correlated Gaussian is a modelling choice, not a fact about atomic physics.

1. **Targets.** TRGB luminosity, helium-core mass at the flash, and surface abundance signatures, each with units and the mass and metallicity range in which it is defined.
2. **Probability model** (Section 3). Priors for $\boldsymbol{a}$ and the other microphysics, including correlations; a likelihood for the data, including systematic floors (Section 1.7).
3. **Verify the forward model.** Convergence of Stellax in mesh and timestep for each regime, including the approach to the helium flash.
4. **Verify the derivatives** (Sections 5.3 and 5.4). Directional finite-difference checks at representative points, on both sides of every event.
5. **Propagate** (Section 1.6). $\mathbf{J}\boldsymbol{\Sigma}_0\mathbf{J}^\mathsf{T}$ first; then an ensemble near the helium flash, where linearization is least trustworthy.
6. **Identifiability** (Section 4). Prior-whitened Fisher eigenvalues; profile likelihoods for the weakly constrained directions; sampling if the posterior is non-Gaussian.
7. **Design** (Section 4.5). Rank candidate observables by EIG with realistic errors and costs.
8. **Connect to observations.** Forward-model instrument response, extinction, distances and selection (Section 3.6.1) rather than comparing theoretical outputs to catalogue values directly.
9. **Check the inference** (Section 3.7). SBC on simulated data before touching real data; posterior predictive checks after.
10. **Alternative physics** (Section 4.4). Repeat with different convection, mass-loss and composition prescriptions.
11. **Qualified claims.** What is constrained within the tested model family, what remains degenerate, and which future data would break the degeneracy.

The result is more than an error bar on one track: a map of which questions about opacity the data can answer.

6.2 Workflow 2: inferring an IMF slope from a cluster

A minimal generative model is

$$M_i \sim p(M \mid \gamma), \qquad \mathbf{y}_i = \mathcal{G}_{\text{evol}}(M_i, \boldsymbol{\psi}), \qquad \mathbf{d}_i \sim p(\mathbf{d} \mid \mathbf{y}_i, \boldsymbol{\zeta}) \text{ if detected,}$$

with shared physics $\boldsymbol{\psi}$ and observing conditions $\boldsymbol{\zeta}$. Extend it with unresolved binaries and with dynamical evolution where relevant: mass segregation and evaporation make the observed stars a biased sample of the initial population.

1. Write the selection-corrected likelihood (Eq. (31)); compute $\alpha(\boldsymbol{\Lambda})$ by pushing simulated populations through the full forward model.
2. Treat a binary fraction and mass-ratio distribution as part of $\boldsymbol{\Lambda}$; expect a posterior correlation with γ.
3. Use gradients with respect to γ through inverse-CDF sampling (Section 5.5). Detection thresholds and remnant boundaries are indicators, so treat each with one of the remedies listed there before trusting its gradient.
4. Sample the joint posterior with HMC (Section 3.5); hierarchical models often need a non-centred parameterization to avoid divergences.
5. Run SBC with populations simulated from the prior before analysing the real cluster.

An optimizer gives a fast first fit. A credible conclusion needs the joint posterior, the selection model, and checks on independent data.

6.3 A recommended progression for a research student

1. A controlled, differentiable TOY model with known answers. Verify every derivative and every propagated uncertainty analytically.
2. A narrow, verified Stellax regime (for example main-sequence models in a small mass range).
3. Calibrated physical uncertainties, with priors you can defend.
4. Local sensitivity and identifiability: Fisher, prior-whitened eigenvalues, profiles.
5. SBC on simulated data.
6. Only then, joint posterior inference on real data and information-optimal design.

This order keeps impressive-looking optimization output from outrunning the numerical and physical evidence behind it.

Exercises for this lecture

Exercises 20, 21, 22, numbered as in Appendix A of the full notes. Worked solutions are in Appendix B; attempt each one before reading its solution.

20. [C] A score-function gradient. For the IMF of Listing 10, estimate $\mathrm{d}P(M > 8\,M_\odot)/\mathrm{d}\gamma$ with the score-function estimator using $\nabla_\gamma \log p(m \mid \gamma)$. Compare its value and its standard error with the exact derivative and with the sigmoid relaxation. How many samples does the score-function estimator need for 5% relative error?

21. **[R] Critique an emulator claim.** A neural network reports 0.1% test error on stellar luminosity, with timesteps from the same tracks split at random between training and test sets. Explain why this may overstate generalization, and design a stronger evaluation, including what you would test if the emulator is later used inside HMC.

22. **[R] Formulate a discovery claim.** You want to constrain an exotic cooling parameter g from TRGB observables in globular clusters. Identify: (i) the target; (ii) at least four nuisance parameters or uncertainty sources; (iii) a likelihood including selection; (iv) an identifiability test; (v) an inference check; (vi) one independent validation.

References

[1] S. Mohamed, M. Rosca, M. Figurnov, and A. Mnih. Monte Carlo gradient estimation in machine learning. *Journal of Machine Learning Research*, 21(132):1–62, 2020.